Relational Database Design & ERD Specification

Normalized Relational Architecture, Schema DDL & Visual Paradigm ERD | Task 3 Deliverable

1. Executive Summary & Problem Scope

This engineering specification details the normalized relational database architecture designed for Walmart Global Tech's Pet Department management system. The persistence model supports complex product taxonomy, animal species mappings, multi-vendor manufacturing, customer transaction histories, and cross-facility logistics tracking across nationwide Walmart fulfillment centers.

The database satisfies all functional and structural constraints in Third Normal Form (3NF):

- **Specialized Product Taxonomy**: Class Table Inheritance (Table-Per-Type) supporting Pet Food, Pet Toys, and Pet Apparel without nullable column sprawl.
- **Many-to-Many Associations**: Associative tables for multi-species animal compatibility (Product_Animal), multi-item customer baskets (Transaction_Product), and multi-SKU freight manifests (Shipment_Product).
- **Supply Chain & Logistics**: Nationwide retail and fulfillment tracking (WalmartLocation) with dual non-identifying foreign keys representing shipment origins and destinations.
- **Referential Integrity**: Cascading lifecycle policies on composition relationships and restrictive foreign keys preventing orphaned records on critical logistics and sales pipelines.

2. Visual Paradigm Standard Entity Relationship Diagram (ERD)

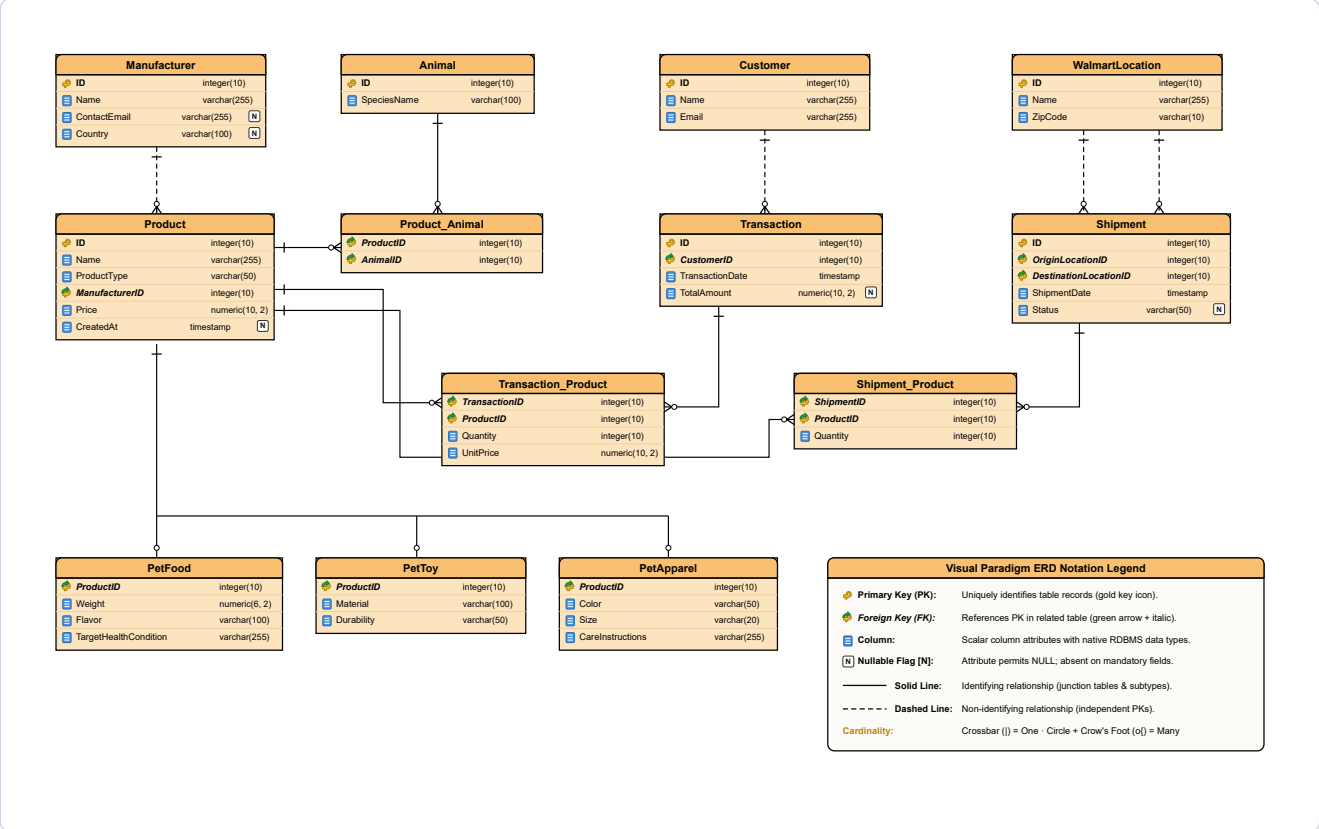


Figure 1: Complete Visual Paradigm standard ERD showing 3NF entities, PK/FK badges, nullability flags, and Crow's foot cardinalities.

3. Relational Schema & Entity Breakdown

Entity Group	Tables	Relational Purpose & Normalization Rationale
Catalog Base	Product, Manufacturer	Base product identity (Name, Price, ProductType) and vendor metadata. Establishes a 1:N non-identifying relationship with Manufacturer.
Subtypes (TPT)	PetFood, PetToy, PetApparel	1:1 Class Table Inheritance. Subtype tables share the primary key of Product (ProductID PK/FK), eliminating NULL column pollution in 3NF.

Animal Mapping	Animal, Product_Animal	Resolves M:N compatibility between products and species (e.g., shared dog/cat harnesses) via composite primary key (ProductID, AnimalID)
Commerce & Sales	Customer, Transaction, Transaction_Product	Captures customer identity, order timestamps, and granular line-item quantities with historical snapshot pricing (UnitPrice).
Logistics & Supply	WalmartLocation, Shipment, Shipment_Product	Tracks origin/destination distribution centers (ZipCode), shipment status lifecycle, and cargo manifests with per-SKU quantity counts.

4. Detailed Requirements Verification Matrix

Initial Prompt Requirement	Modeled Relational Entities	Implementation Details & Constraints
Pet Food: name, manufacturer, weight, flavor, target health condition	Product + PetFood	Weight numeric(6,2), Flavor varchar(100), TargetHealthCondition varchar(255). References Product.ID.
Pet Toys: material, name, manufacturer, durability	Product + PetToy	Material varchar(100), Durability varchar(50). References Product.ID.
Pet Apparel: color, manufacturer, size, name, care instructions	Product + PetApparel	Color varchar(50), Size varchar(20), CareInstructions varchar(255). References Product.ID.
Associated with 1+ animals	Animal + Product_Animal	M:N junction with composite key (ProductID, AnimalID). Supports multi-animal tagging.
Associated with a manufacturer	Manufacturer + Product	1:N relationship via Product.ManufacturerID foreign key with mandatory NOT NULL constraint.
Customers: names and emails	Customer	Name varchar(255), Email varchar(255) with UNIQUE constraint and indexing.
Customer transactions: date, products	Transaction + Transaction_Product	1:N Customer-to-Transaction, M:N Transaction-to-Product via junction with Quantity and UnitPrice.
Walmart locations: name, zip code	WalmartLocation	Name varchar(255), ZipCode varchar(10) format validated.
Shipments: origin, destination, products & quantity	Shipment + Shipment_Product	Dual foreign keys to WalmartLocation (OriginLocationID, DestinationLocationID), M:N cargo manifest junction.

5. Physical DDL Implementation Highlights

The production schema (schema.sql) implements the following database engineering practices:

- **PostgreSQL / ANSI SQL-92 Compliance**: Utilizes standard sequence generators (SERIAL), exact numeric scales (DECIMAL(10,2)), and ISO timestamps.
- **Referential Actions**: Employs ON DELETE CASCADE for parent-dependent records (e.g. removing a Product cascades to its subtype attributes and junction records) and ON DELETE RESTRICT for financial transactions and audit logs.
- **Check Constraints**: Ensures business logic consistency directly at the storage layer, including origin-destination disparity (CHECK (origin_location_id <> destination_location_id)) and positive quantity bounds (CHECK (quantity > 0)).